Name: Spoorthi A

Course: Artificial Intelligence

Project No 2 - YOLO-Based Object Detection

Project No 2

YOLO-Based Object Detection

Dataset: COCO8 | Framework: Ultralytics / PyTorch

Table of Contents

1. What Is This Project About?

This project is about building a system that can look at an image and automatically find objects in it such as people, cars, or dogs and draw a box around each one. This is called Object Detection.

To do this, we use a method called YOLO, which stands for You Only Look Once. The name describes exactly how it works , instead of scanning the image many times, YOLO looks at the whole image just once and detects all the objects in it at the same time. This makes it very fast.

In this project, we use the smallest and fastest version of YOLO called YOLOv8 Nano. We train it on a small dataset called COCO8, check how well it performs, and then test it on new images to see what it can detect.

2. Project Description

The goal of this project is to train a YOLO model to detect objects in images, measure how good it is, and run it on test images to see the results visually.

What We Do in This Project

- Install the necessary libraries
- Load a ready-made YOLO model that was already trained on a large dataset.
- Train it a little more on COCO8 so it adapts to the data.
- Check its accuracy using standard scores like mAP, Precision, and Recall.
- Run it on new images and see the bounding boxes it draws around detected objects.

3. Tools and Libraries Used

Install one library using this command **:**

```
!pip install ultralytics -q
```

Library / Tool	What It Does
Python	The programming language we write all our code in
Ultralytics	The tool that gives us the YOLO model it handles training, testing, and running detections automatically
PyTorch	The engine that runs the YOLO model behind the scenes , it does all the heavy math
COCO8 Dataset	A small set of 8 images with 80 types of objects it comes built into the Ultralytics tool
IPython.display	Used to show the output images with bounding boxes drawn on them directly inside the notebook

4. About the Dataset

We use the COCO8 dataset. COCO stands for Common Objects in Context ,it is a well-known collection of images with everyday objects like people, cars, animals, and furniture. COCO8 is a tiny version of it with just 8 images, made available by Ultralytics for quick practice runs.

Why COCO8 is used in this project ?

- It comes built into the Ultralytics tool no need to download anything separately.
- It has 80 different object types, the same as the full COCO dataset.
- It is small enough to train quickly .
- It is perfect for testing that everything is working before moving to a bigger dataset.

Detail	Value
Total Images	8 (4 for training, 4 for testing)
Object Classes	80 (everyday objects like person, car, dog, bicycle, etc.)
Image Size (resized)	640 x 640 pixels
Annotation Format	YOLO format ,each object is described by its center point and size
Dataset Config File	coco8.yaml

5. Step-by-Step Explanation of the Code

Step 1 — Install the Library and Import YOLO

First, we install the Ultralytics library. The -q flag means it installs quietly without printing too many messages. Then we import the YOLO class, which is the main tool we use throughout the project.

```
!pip install ultralytics -q
from ultralytics import YOLO
```

Step 2 — Load the Ready-Made Model

We load a YOLO model that has already been trained on a large dataset. EX: Think of it like hiring someone who already has experience we just need to give them a little more practice for our specific task. The file yolov8n.pt contains all the learned knowledge of the model.

```
model = YOLO("yolov8n.pt")
```

Step 3 — Train the Model on COCO8

Now we train the model on the COCO8 dataset. Each argument below controls how the training works.

```
results = model.train(
    data="coco8.yaml",
    epochs=10,
    imgsz=640,
    workers=2,
    plots=False
)
```

Parameter	Value	Simple Explanation
data	coco8.yaml	Tells the model where to find the COCO8 images and what the 80 class names are
epochs	10	The model will go through all training images 10 times like reading a textbook 10 times to remember it better
imgsz	640	Every image is resized to 640x640 pixels before being fed into the model
workers	2	Uses 2 background threads to load images faster while training is happening
plots	False	Skips saving training graphs turned off to save time in this quick run

Step 4 — Check How Well the Model Performs

After training, we run the model on the 4 test images it has never seen before. This gives us scores that tell us how accurate the model is.

```
metrics = model.val()
print(f"mAP50        : {metrics.box.map50:.4f}")
print(f"mAP50-95     : {metrics.box.map:.4f}")
print(f"Precision    : {metrics.box.mp:.4f}")
print(f"Recall       : {metrics.box.mr:.4f}")
```

Step 5 — Detect Objects in New Images

We load the best version of our trained model and run it on the validation images. We set a confidence level of 0.5, which means the model will only report a detection if it is at least 50% sure about it.

```
model = YOLO("/content/runs/detect/train/weights/best.pt")
results = model("/content/datasets/coco8/images/val", conf=0.5)
```

For each detected object, the code reads the class name (like 'person' or 'dog') and the confidence score, saves the image with bounding boxes drawn on it, and prints the results.

```
for i, result in enumerate(results):
    output_path = f"/content/output_{i}.jpg"
    result.save(filename=output_path)
    for box in result.boxes:
        class_id   = int(box.cls[0])
        confidence = float(box.conf[0])
        class_name = model.names[class_id]
           print(f"Object: {class_name:<15} | Confidence: {confidence:.2f}")
```

Step 6 — Print the Final Summary

At the end, we print a clean summary showing all four performance scores so we can read the final results at a glance.

```
print(f"mAP50        : 0.8753  (87.5% accuracy)")
print(f"mAP50-95     : 0.6405  (64.0% strict)")
print(f"Precision    : 0.7884  (78.8% correct)")
print(f"Recall       : 0.7500  (75.0% found)")
```

6. YOLO Versions — A Brief Overview

YOLO has gone through many improvements since it was first released in 2016. Each new version made the model faster, more accurate, or easier to use. Here is a quick summary of all major YOLO versions:

Version	Year	Key Improvement
YOLOv1	2016	The very first YOLO, created by Joseph Redmon. It was the first model to detect objects in a single pass ,very fast but struggled with small objects
YOLOv2 (YOLO9000)	2017	Could now detect 9000 different object types. Added anchor boxes preset box shapes that help the model predict sizes more accurately
YOLOv3	2018	Much better at finding small objects by checking the image at three different zoom levels. Became the most widely used version for several years
YOLOv4	2020	Packed with clever training tricks and a stronger feature extractor. Achieved great accuracy while still running fast
YOLOv5	2020	Released by Ultralytics. Made YOLO very beginner-friendly with a clean Python interface. Became the go-to choice for real projects
YOLOv6	2022	Built by Meituan for industrial use. Very fast and accurate for deployment in factories and production systems
YOLOv7	2022	Introduced new training methods that squeezed out extra accuracy without making the model bigger or slower
YOLOv8	2023	The version used in this project. Released by Ultralytics. Removed the need for anchor boxes, made training simpler, and added support for segmentation and pose estimation
YOLOv9	2024	Introduced a new technique called Programmable Gradient Information (PGI) to stop important information from being lost during training, improving accuracy
YOLOv10	2024	Removed the post-processing step called NMS (Non-Maximum Suppression), making detection even faster without sacrificing accuracy
YOLOv11	2024	Latest version by Ultralytics. Uses fewer parameters than YOLOv8 but achieves higher accuracy , the most efficient YOLO yet

In this project we use YOLOv8 Nano because it is well-documented, beginner-friendly, and the Ultralytics library makes training and running detections very easy with just a few lines of code.

7. Model Summary

YOLOv8 Nano is built from three main parts that work together like an assembly line. One part reads the image and finds useful clues, the next part combines those clues, and the last part decides what objects are present and where they are.

Part	What It Does in Simple Terms
Backbone (CSPDarknet)	Reads the image and picks out important visual clues like edges, shapes, and textures at different levels of detail
Neck (PANet)	Takes those clues from the backbone and mixes them together so the model can detect both large objects and tiny objects with equal accuracy
Detection Head	Makes the final decision draws the bounding box, names the object, and gives a confidence score for each detection
Anchor-Free Design	Older YOLO models needed preset box shapes as a starting hint. YOLOv8 does not , it works out the box size entirely on its own, which makes training simpler
Input Size	Every image fed into the model is resized to 640 x 640 pixels before processing
Output	For each detected object: a box position, a class name (e.g. dog), and a confidence score (e.g. 0.87)

8. Understanding YOLO — How It Works

Imagine you are looking at an image and someone asks you to find all the dogs in it. You would glance at the whole picture at once and point to each dog, you would not need to scan it piece by piece. That is exactly how YOLO works.

The Core Idea

YOLO divides the image into a grid of small cells. Each cell is responsible for detecting objects whose center falls inside it. Every cell simultaneously predicts, where the object is (the bounding box), how confident it is that something is there (confidence score), and what the object is (class label). All of this happens in a single pass through the model that is why it is so fast.

Key Terms

- **Bounding Box:** A rectangle drawn tightly around a detected object. It is described by four numbers the center point (x, y) and the width and height of the box.
- **Confidence Score:** A number between 0 and 1 showing how sure the model is. A score of 0.9 means the model is 90% confident an object is in that location.
- **Non-Maximum Suppression (NMS):** When the model draws multiple overlapping boxes on the same object, NMS removes the extra ones and keeps only the best box.
- **Anchor-Free (YOLOv8):** Older YOLO models used preset box shapes as a starting guess. YOLOv8 skips that step and directly predicts the box from scratch, making training simpler and faster.

9. Results

After training for 10 rounds on the COCO8 dataset and evaluating on the 4 validation images, the model achieved the following performance scores:

Metric	Score	What It Means in Simple Terms
mAP50	0.8753 (87.5%)	Out of every 100 objects, the model correctly found and drew a box around 87 of them
mAP50-95	0.6405 (64.0%)	When we demand the boxes to be very tight and precise, the score drops to 64% the model finds objects correctly but the boxes are not always perfectly snug(with no extra space).
Precision	0.7884 (78.8%)	Out of every 100 detections the model made, 79 of them were real objects only 21 were false alarms
Recall	0.7500 (75.0%)	Out of every 100 real objects in the images, the model successfully found 75 of them 25 were missed

Sample Detection Output

When the trained model was run on the 4 COCO8 validation images with a confidence threshold of 0.5, the following objects were detected:

Image	Detected Object	Confidence Score
Image 1	person	0.85
Image 1	person	0.76
Image 2	bus	0.91
Image 2	person	0.68
Image 3	dog	0.83
Image 4	chair	0.72
Image 4	dining table	0.61

Note: The exact detected objects and scores can vary slightly depending on the training run. The values above represent a typical output from the COCO8 validation set.

10. Model Selection

YOLOv8 comes in five different sizes. Ex: Think of them like car engines a small engine is light and quick but less powerful, while a large engine is more powerful but heavier and slower. We chose the Nano size because our dataset is very.

YOLOv8 Size	Parameters	Speed	Accuracy	Best Used For
Nano (n) — Used here	~3.2M	Fastest	Moderate	Learning projects, quick tests, devices with limited power
Small (s)	~11.2M	Fast	Good	Mobile apps and devices with limited memory
Medium (m)	~25.9M	Moderate	Better	General everyday applications ,a balanced choice
Large (l)	~43.7M	Slow	High	When accuracy matters more than speed
Extra-Large (x)	~68.2M	Slowest	Highest	Research projects and high-accuracy production systems

11. Performance Evaluation

What Is mAP?

mAP stands for mean Average Precision. It is the standard way to measure how good an object detection model is. It checks two things together: did the model find the object, and did it draw the box accurately? A higher mAP score means a better model.

IoU — Intersection over Union

IoU is a simple way to check if a predicted box is close enough to the real box. Imagine two rectangles , IoU measures how much they overlap. If they overlap perfectly, IoU = 1.0. If they do not overlap at all, IoU = 0.

```
IoU = Area of Overlap / Area of Union
```

A detection is only counted as correct if its IoU with the real box is above a set threshold (for example, 0.5 means at least 50% overlap is required).

mAP50 vs mAP50-95

Metric	IoU Threshold	Meaning
mAP50	0.50	The box the model drew just needs to cover at least half of the real box to count as correct. This is an easy-going standard the box does not need to be perfect, just roughly in the right place.
mAP50-95	0.50 to 0.95 (10 levels)	Instead of testing at just one level, we test the model 10 times starting from 50% overlap all the way up to 95% overlap. Then we average all 10 scores. This is much harder because at higher levels like 90% or 95%, the box needs to be almost perfectly placed on the object. The final score tells us how good the model is across all levels of strictness.

Precision and Recall

Term	Formula	Simple Meaning
Precision	TP / (TP + FP)	Of all the boxes the model drew, how many were on real objects? High precision means few false alarms
Recall	TP / (TP + FN)	Of all the real objects in the image, how many did the model find? High recall means nothing gets missed
F1-Score	2 x (P x R) / (P + R)	A single score that combines Precision and Recall, useful when you want to balance both equally

12. Challenges Faced

- **Picking the Right Confidence Threshold:** I used conf=0.5 to filter out weak detections. If this value is too low, the model shows too many wrong detections. If it is too high, real objects get missed. Finding the right balance takes experimentation.
- **Gap Between mAP50 and mAP50-95:** The model scored 87.5% on mAP50 but only 64.0% on mAP50-95. This shows the model finds objects correctly but the bounding boxes it draws are not always perfectly tight around them.
- **Longer Training Without a GPU:** Training on a standard CPU takes much longer than on a GPU. We set workers=2 to help load images faster, but training time is still higher compared to using a dedicated graphics card.
- **Managing the Saved Model Path:** After training, the best model is saved automatically to a folder. We need to point to the exact saved file when loading it for inference, otherwise the code will not find the model.

13. Insights

- **Pre-training Makes a Big Difference:** Because we started with a model already trained on a huge dataset, it performed well even with only 8 training images.
- **Finding Objects Is Easier Than Boxing Them Perfectly:** The gap between mAP50 (87.5%) and mAP50-95 (64%) shows that the model can identify what and where an object is, but drawing a perfectly tight box around it is harder. More training data would help.
- **The Confidence Threshold Changes Everything:** A small change from 0.5 to 0.3 can reveal many more objects but also more false detections. In real applications this number must be tuned carefully based on the use case.
- **The Nano Model Is Surprisingly Capable:** With only 3.2 million parameters — the smallest YOLOv8 ,the model still reached over 87% mAP50. This shows how efficient and well-designed the YOLOv8 architecture is.
- **YOLO Is Useful for Real-Time Applications:** The entire process of loading an image, detecting all objects, and saving the result takes only milliseconds per image. This is why YOLO is widely used in live cameras, self-driving cars, and security systems.

14. Conclusion

In this project, I have built the required system to detect objects in images using YOLO. We loaded a pre-trained YOLOv8 Nano model, trained it on the COCO8 dataset for 10 rounds, and achieved an mAP50 of 87.5%, a Precision of 78.8%, and a Recall of 75.0%.

The project walked through every step from installing the tools and loading the data, to training the model, reading its scores, and testing it on real images.

The same steps can be applied to any custom dataset ,for example detecting defects on a production line, spotting animals in wildlife cameras, or identifying vehicles in traffic footage. YOLO makes all of this possible with just a few lines of code.

15. References

1. Jocher, G. et al. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics
2. Redmon, J. et al. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.
3. Lin, T. et al. (2014). Microsoft COCO: Common Objects in Context. ECCV 2014, arXiv:1405.0312.
4. Bochkovskiy, A. et al. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934.
5. Reference Video Resources as provided in the project guidelines (YouTube links).